



**RV College of Engineering®**

Mysore Road, RV Vidyaniketan Post,  
Bengaluru - 560059, Karnataka, India

## Department of Computer Science and Engineering

|                                           |          |                      |
|-------------------------------------------|----------|----------------------|
| Date: 12/01/2026                          | Test - 3 | Max. Marks:60        |
| Semester: 5                               | UG       | Duration: 2 Hrs      |
| Course Title: DATABASE MANAGEMENT SYSTEMS |          | Course Code: CD252IA |

| S. No. | Part A (Questions)                                                                             | M | BT | CO  |
|--------|------------------------------------------------------------------------------------------------|---|----|-----|
| 1      | List and define the ACID properties of database transactions.                                  | 2 | 1  | CO1 |
| 2      | List the three conditions to identify that the two Transactions are conflicting to each other. | 2 | 1  | CO1 |
| 3      | Mention the importance of Commit and Rollback.                                                 | 2 | 2  | CO2 |
| 4      | Distinguish between Master-Slave replication and Peer-to-Peer replication.                     | 2 | 1  | CO1 |
| 5      | Differentiate between SQL and NoSQL database.                                                  | 2 | 2  | CO3 |

| S. No. | Part B (Questions)                                                                                                                                                                                                                                                                                                                                                                                                                                     | M  | BT | CO  |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|----|-----|
| 1      | Consider the relation $R = (A_1, A_2, A_3, A_4, A_5)$ and the set of functional dependencies<br>$FD = \{A_1 \rightarrow A_3A_5, A_5 \rightarrow A_1A_4, A_3A_4 \rightarrow A_2\}$ .<br>Determine whether the decomposition<br>$D = \{R_1, R_2, R_3\}$ , where<br>$R_1 = \{A_1, A_2, A_3, A_5\}$ ,<br>$R_2 = \{A_1, A_3, A_4\}$ , and<br>$R_3 = \{A_4, A_5\}$ ,<br>is a lossless join decomposition. Show all the necessary steps as per the algorithm. | 10 | 4  | CO3 |
| 2      | Define concurrency. Explain the need for concurrent execution. List and describe the various problems encountered due to concurrent execution with relevant examples.                                                                                                                                                                                                                                                                                  | 10 | 2  | CO2 |



| 3        | a) What is transaction? Explain the states of transaction execution with a neat diagram.                                                                                                                                                                                           | 5        | 2  | CO1 |  |  |         |  |         |  |  |  |          |         |  |  |          |  |  |  |  |  |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|----|-----|--|--|---------|--|---------|--|--|--|----------|---------|--|--|----------|--|--|--|--|--|
|          | b) Consider the following schedule of transactions.                                                                                                                                                                                                                                | 5        | 2  | CO3 |  |  |         |  |         |  |  |  |          |         |  |  |          |  |  |  |  |  |
|          | <table><tr><th>T1</th><th>T2</th><th>T3</th></tr><tr><td></td><td></td><td>Read(A)</td></tr><tr><td></td><td>Read(A)</td><td></td></tr><tr><td></td><td></td><td>Write(A)</td></tr><tr><td>Read(A)</td><td></td><td></td></tr><tr><td>Write(A)</td><td></td><td></td></tr></table> | T1       | T2 | T3  |  |  | Read(A) |  | Read(A) |  |  |  | Write(A) | Read(A) |  |  | Write(A) |  |  |  |  |  |
| T1       | T2                                                                                                                                                                                                                                                                                 | T3       |    |     |  |  |         |  |         |  |  |  |          |         |  |  |          |  |  |  |  |  |
|          |                                                                                                                                                                                                                                                                                    | Read(A)  |    |     |  |  |         |  |         |  |  |  |          |         |  |  |          |  |  |  |  |  |
|          | Read(A)                                                                                                                                                                                                                                                                            |          |    |     |  |  |         |  |         |  |  |  |          |         |  |  |          |  |  |  |  |  |
|          |                                                                                                                                                                                                                                                                                    | Write(A) |    |     |  |  |         |  |         |  |  |  |          |         |  |  |          |  |  |  |  |  |
| Read(A)  |                                                                                                                                                                                                                                                                                    |          |    |     |  |  |         |  |         |  |  |  |          |         |  |  |          |  |  |  |  |  |
| Write(A) |                                                                                                                                                                                                                                                                                    |          |    |     |  |  |         |  |         |  |  |  |          |         |  |  |          |  |  |  |  |  |
|          | i. Construct the precedence graph for the given schedule.                                                                                                                                                                                                                          |          |    |     |  |  |         |  |         |  |  |  |          |         |  |  |          |  |  |  |  |  |
|          | ii. Check whether the above schedule is conflict-serializable or not. If it is conflict-serializable, write the equivalent serial schedule.                                                                                                                                        |          |    |     |  |  |         |  |         |  |  |  |          |         |  |  |          |  |  |  |  |  |
| 4        | a) Explain the Column-Family Stores aggregate data model with a neat diagram.                                                                                                                                                                                                      | 5        | 3  | CO3 |  |  |         |  |         |  |  |  |          |         |  |  |          |  |  |  |  |  |
|          | b) Differentiate between sharding and replication. Explain how sharding and replication can be combined, with a neat diagram.                                                                                                                                                      | 5        | 2  | CO1 |  |  |         |  |         |  |  |  |          |         |  |  |          |  |  |  |  |  |
| 5        | a) Classify digital data with a neat diagram. Differentiate between structured, semi-structured, and unstructured data.                                                                                                                                                            | 5        | 2  | CO1 |  |  |         |  |         |  |  |  |          |         |  |  |          |  |  |  |  |  |
|          | b) Define Hadoop. Explain the key considerations of Hadoop with a neat diagram.                                                                                                                                                                                                    | 5        | 2  | CO1 |  |  |         |  |         |  |  |  |          |         |  |  |          |  |  |  |  |  |

## CIE 3: Database Management Systems (DBMS)

**Date:** 12th January 2026

**Assessment:** Quiz

### 1. List and define the ACID properties of database transactions.

The ACID properties ensure that database transactions are processed reliably.

- **Atomicity:** This property ensures that a transaction is treated as a single unit of work. Either all operations within the transaction are completed successfully, or none of them are. If any part fails, the entire transaction is aborted (all or nothing).
- **Consistency:** This ensures that a transaction brings the database from one valid state to another, maintaining all defined rules, constraints, and cascades. The data must satisfy all integrity constraints before and after the transaction.
- **Isolation:** This ensures that concurrently executing transactions do not interfere with each other. The intermediate state of a transaction should not be visible to other transactions until it is committed.
- **Durability:** This guarantees that once a transaction has been committed, it will remain so, even in the event of a power loss, crash, or error. The changes are permanently saved to non-volatile storage.

### 2. List the 3 conditions to identify that 2 transactions are conflicting to each other.

Two operations from different transactions are considered conflicting if they satisfy **all** of the following three conditions:

1. **Different Transactions:** The operations belong to two different transactions.
2. **Same Data Item:** Both operations access the exact same data item (e.g., both access row X ).
3. **At Least One Write:** At least one of the operations is a write operation ( `write(X)` ).
  - *Conflict pairs:* (Read, Write), (Write, Read), (Write, Write).
  - *(Read, Read) is NOT a conflict.*

### 3. Mention the importance of COMMIT and ROLLBACK.

These commands are essential for Transaction Control Language (TCL) to maintain data integrity and Atomicity.

- **COMMIT:**
  - **Importance:** It permanently saves all changes made during the current transaction to the database. Once executed, the changes are visible to other users and cannot be undone. It marks the successful end of a transaction.

- **ROLLBACK:**

- **Importance:** It undoes all changes made during the current transaction, reverting the database to the state it was in before the transaction started (or to the last Savepoint). It is crucial for handling errors and ensuring that partial or failed transactions do not corrupt the data.

#### 4. Distinguish between Master-Slave Replication and Peer-to-Peer Replication.

| Feature                 | Master-Slave Replication                                    | Peer-to-Peer (Master-Master) Replication                          |
|-------------------------|-------------------------------------------------------------|-------------------------------------------------------------------|
| <b>Write Operations</b> | Only the <b>Master</b> node handles write requests.         | <b>All nodes</b> can handle write requests.                       |
| <b>Read Operations</b>  | Slaves generally handle read requests (scaling reads).      | All nodes can handle read requests.                               |
| <b>Data Flow</b>        | Unidirectional: Master copies data to Slaves.               | Bidirectional/Multi-directional: Changes propagate to all peers.  |
| <b>Consistency</b>      | Easier to maintain consistency (single source of truth).    | Harder to maintain; requires complex conflict resolution.         |
| <b>Availability</b>     | If Master fails, writes stop until a new Master is elected. | High availability for writes; if one node fails, others continue. |

#### 5. Differentiate between SQL and NoSQL database.

| Feature               | SQL (Relational)                                                           | NoSQL (Non-Relational)                                                                                              |
|-----------------------|----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>Structure</b>      | Table-based with a predefined schema.                                      | Document, Key-Value, Graph, or Wide-column stores.                                                                  |
| <b>Schema</b>         | <b>Rigid/Static:</b> Structure must be defined before adding data.         | <b>Flexible/Dynamic:</b> Structure can change on the fly.                                                           |
| <b>Scaling</b>        | <b>Vertical Scaling</b> (Scale-up): Increasing server hardware (RAM, CPU). | <b>Horizontal Scaling</b> (Scale-out): Adding more servers to the pool.                                             |
| <b>Query Language</b> | Uses <b>Structured Query Language (SQL)</b> standard.                      | Uses proprietary or non-standard interfaces (UnQL, API calls).                                                      |
| <b>Transactions</b>   | Strong <b>ACID</b> compliance.                                             | Often follows <b>BASE</b> (Basically Available, Soft state, Eventual consistency) model (though some support ACID). |
| <b>Examples</b>       | MySQL, PostgreSQL, Oracle, SQL Server.                                     | MongoDB, Cassandra, Redis, Neo4j.                                                                                   |

## DBMS Test Solution: Lossless Join Decomposition

Marks: 10

Topic: Relational Database Design / Normalization

### Problem Statement

Relation:  $R = (A_1, A_2, A_3, A_4, A_5)$

### Functional Dependencies (FD):

1.  $A_1 \rightarrow A_3 A_5$
2.  $A_5 \rightarrow A_1 A_4$
3.  $A_3 A_4 \rightarrow A_2$

Decomposition:  $D = \{R_1, R_2, R_3\}$

- $R_1 = (A_1, A_2, A_3, A_5)$
- $R_2 = (A_1, A_3, A_4)$
- $R_3 = (A_4, A_5)$

**Question:** Determine whether the decomposition  $D$  is a lossless join decomposition. Show all necessary steps using the Matrix (Chase) Algorithm.

### Solution: The Chase Algorithm

To test for lossless join, we construct a matrix  $S$  with rows corresponding to the relations in the decomposition  $(R_1, R_2, R_3)$  and columns corresponding to the attributes of  $R$  ( $A_1$  to  $A_5$ ).

**Goal:** If, after applying the FDs, any row in the matrix becomes all  $a$ 's ( $a_1, a_2, a_3, a_4, a_5$ ), the decomposition is **Lossless**.

### Step 1: Initialization

Fill the matrix. If an attribute  $A_j$  is present in relation  $R_i$ , put  $a_j$ . Otherwise, put a distinct symbol  $b_{ij}$ .

- **Row 1 ( $R_1$ ):** Has  $A_1, A_2, A_3, A_5$ . Missing  $A_4$  ( $b_{14}$ ).
- **Row 2 ( $R_2$ ):** Has  $A_1, A_3, A_4$ . Missing  $A_2$  ( $b_{22}$ ),  $A_5$  ( $b_{25}$ ).
- **Row 3 ( $R_3$ ):** Has  $A_4, A_5$ . Missing  $A_1$  ( $b_{31}$ ),  $A_2$  ( $b_{32}$ ),  $A_3$  ( $b_{33}$ ).

Initial Matrix  $S_0$ :

|       | $A_1$ | $A_2$ | $A_3$ | $A_4$    | $A_5$ |
|-------|-------|-------|-------|----------|-------|
| $R_1$ | $a_1$ | $a_2$ | $a_3$ | $b_{14}$ | $a_5$ |

|       |          |          |          |       |          |
|-------|----------|----------|----------|-------|----------|
| $R_2$ | $a_1$    | $b_{22}$ | $a_3$    | $a_4$ | $b_{25}$ |
| $R_3$ | $b_{31}$ | $b_{32}$ | $b_{33}$ | $a_4$ | $a_5$    |

Step 2: Apply FDs recursively

Apply FD 1:  $A_1 \rightarrow A_3, A_5$

- Check column  $A_1$ . Rows 1 and 2 both have value  $a_1$ .
- Therefore, they must agree on columns  $A_3$  and  $A_5$ .
- **Column  $A_3$ :** Both are already  $a_3$ . No change.
- **Column  $A_5$ :** Row 1 has  $a_5$ , Row 2 has  $b_{25}$ . We replace  $b_{25}$  with  $a_5$ .

Matrix  $S_1$ :

|       |          |          |          |          |       |
|-------|----------|----------|----------|----------|-------|
|       | $A_1$    | $A_2$    | $A_3$    | $A_4$    | $A_5$ |
| $R_1$ | $a_1$    | $a_2$    | $a_3$    | $b_{14}$ | $a_5$ |
| $R_2$ | $a_1$    | $b_{22}$ | $a_3$    | $a_4$    | $a_5$ |
| $R_3$ | $b_{31}$ | $b_{32}$ | $b_{33}$ | $a_4$    | $a_5$ |

Apply FD 2:  $A_5 \rightarrow A_1, A_4$

- Check column  $A_5$ . Rows 1, 2, and 3 all have value  $a_5$ .
- Therefore, they must agree on columns  $A_1$  and  $A_4$ .
- **Column  $A_1$ :** Rows 1 & 2 have  $a_1$ . Row 3 has  $b_{31}$ . Replace  $b_{31}$  with  $a_1$ .
- **Column  $A_4$ :** Rows 2 & 3 have  $a_4$ . Row 1 has  $b_{14}$ . Replace  $b_{14}$  with  $a_4$ .

Matrix  $S_2$ :

|       |       |          |          |       |       |
|-------|-------|----------|----------|-------|-------|
|       | $A_1$ | $A_2$    | $A_3$    | $A_4$ | $A_5$ |
| $R_1$ | $a_1$ | $a_2$    | $a_3$    | $a_4$ | $a_5$ |
| $R_2$ | $a_1$ | $b_{22}$ | $a_3$    | $a_4$ | $a_5$ |
| $R_3$ | $a_1$ | $b_{32}$ | $b_{33}$ | $a_4$ | $a_5$ |

Step 3: Verification

Check if any row contains purely  $a$  values ( $a_1, a_2, a_3, a_4, a_5$ ).

Looking at **Row 1 ( $R_1$ )** in Matrix  $S_2$ :

- $A_1$ :  $a_1$

- $A_2: a_2$
- $A_3: a_3$
- $A_4: a_4$
- $A_5: a_5$

Row 1 consists entirely of  $a$  symbols.

*(Note: We do not need to apply the third FD,  $A_3A_4 \rightarrow A_2$ , because we have already found a complete row, but applying it would also turn Row 2 into all  $a$ 's, confirming the result further).*

### Conclusion

Since Row 1 of the matrix has become  $(a_1, a_2, a_3, a_4, a_5)$ , which represents the original relation tuple, the decomposition  $D$  is **LOSSLESS**.

# DBMS Question: Concurrency & Associated Problems

**Marks:** 10

**Topic:** Transaction Management / Concurrency Control

## 1. Definition of Concurrency

**Concurrency** in a Database Management System (DBMS) refers to the execution of multiple transactions simultaneously in a shared database environment.

In a multi-programming environment, multiple transactions can be executed at the same time. The database scheduler interleaves the operations of these transactions to give the illusion that they are executing simultaneously, or they may actually execute in parallel on multi-core processors. This ensures that the database is utilized efficiently.

## 2. Need for Concurrent Execution

Executing transactions serially (one after another) is inefficient. Concurrent execution is essential for the following reasons:

1. **Improved Throughput:** Throughput is the number of transactions executed per unit of time. By allowing multiple transactions to run simultaneously, the CPU and I/O disks can work in parallel, significantly increasing the total number of transactions processed.
2. **Reduced Waiting Time (Response Time):** If transactions are executed serially, a short transaction might have to wait for a very long transaction to complete. Concurrency allows short transactions to finish quickly without waiting for long processes, improving the system's responsiveness.
3. **Better Resource Utilization:** While one transaction is waiting for an I/O operation (reading from disk), the CPU can process another transaction. This ensures that system resources (CPU and Disk) are not idle.

## 3. Problems Encountered Due to Concurrent Execution

When multiple transactions access and modify the same data items without proper control (Isolation), several data inconsistency problems arise.

### A. The Lost Update Problem (Write-Write Conflict)

This occurs when two transactions read the same data item and then update it based on the value read. The update performed by the first transaction is overwritten (lost) by the update of the second transaction.

**Example:** Consider a bank account A with a balance of \$1000.

- **T1:** Withdraws \$100.
- **T2:** Withdraws \$200.



| Time | Transaction T1         | Transaction T2         | Value of A in DB |
|------|------------------------|------------------------|------------------|
| 1    | Read(A) // Reads 1000  |                        | 1000             |
| 2    |                        | Read(A) // Reads 1000  | 1000             |
| 3    | A = A - 100            |                        | 1000             |
| 4    | Write(A) // Writes 900 |                        | 900              |
| 5    |                        | A = A - 200            | 900              |
| 6    |                        | Write(A) // Writes 800 | <b>800</b>       |

**Result:** The final balance is \$800, but it should be \$700 (\$1000 - 100 - 200). T1's update was lost.

### B. The Dirty Read Problem (Write-Read Conflict)

Also known as the **Uncommitted Dependency** problem. This occurs when a transaction reads a data item that has been updated by another transaction that has not yet committed. If the other transaction fails and rolls back, the first transaction has read "dirty" (invalid) data.

**Example:** Consider account A with balance \$1000.

| Time | Transaction T1             | Transaction T2        | Value of A  |
|------|----------------------------|-----------------------|-------------|
| 1    | Read(A)                    |                       | 1000        |
| 2    | A = A + 500                |                       |             |
| 3    | Write(A) // Writes 1500    |                       | 1500        |
| 4    |                            | Read(A) // Reads 1500 | 1500        |
| 5    |                            | Commit                |             |
| 6    | <b>Rollback</b> (T1 Fails) |                       | <b>1000</b> |

**Result:** T2 read the value 1500 and committed. However, T1 failed and rolled back the value to 1000. T2 worked with a value that never technically existed in the permanent database state.

### C. The Unrepeatable Read Problem (Read-Write Conflict)

This occurs when a transaction reads the same variable twice during its execution. Between the two reads, another transaction updates the value. Consequently, the first transaction gets different values for the same variable in the same transaction execution.

**Example:** Consider account A with balance \$1000.

| Time | Transaction T1        | Transaction T2          | Value of A |
|------|-----------------------|-------------------------|------------|
| 1    | Read(A) // Reads 1000 |                         | 1000       |
| 2    |                       | Read(A)                 | 1000       |
| 3    |                       | $A = A + 200$           |            |
| 4    |                       | Write(A) // Writes 1200 | 1200       |
| 5    | Read(A) // Reads 1200 |                         | 1200       |

**Result:** T1 read \$1000 at Time 1 and \$1200 at Time 5. This inconsistency can cause logical errors in calculations requiring a stable snapshot of the data.

#### D. The Phantom Read Problem

This occurs when a transaction executes a query that retrieves a set of rows satisfying a certain condition. While the transaction is running, another transaction inserts or deletes a row that satisfies that condition. When the first transaction re-executes the query, it finds a different set of rows (a "phantom" row appears or disappears).

**Example:**

- **T1:** SELECT \* FROM Employee WHERE Salary > 50000; (Returns 5 rows).
- **T2:** INSERT INTO Employee (Name, Salary) VALUES ('John', 60000); (Commits).
- **T1:** Re-executes SELECT \* FROM Employee WHERE Salary > 50000; (Now returns 6 rows).

**Result:** The result set changed during the transaction, causing inconsistency.

## DBMS Question 3: Transactions & Serializability

Marks: 10

Topic: Transaction Management / Concurrency Control

### 3. a) Transaction Definition and States

#### Definition

A **Transaction** is a single logical unit of work that accesses and possibly modifies the contents of a database. Transactions access data using read and write operations. To maintain data integrity, a transaction must satisfy the ACID properties (Atomicity, Consistency, Isolation, Durability).

#### States of Transaction Execution

A transaction must be in one of the following states during its execution lifecycle:

1. **Active:** The initial state. The transaction stays in this state while it is executing read and write operations.
2. **Partially Committed:** The transaction enters this state after the final statement has been executed, but before the changes are made permanent.
3. **Failed:** The transaction enters this state if the normal execution can no longer proceed (due to hardware failure or logical error).
4. **Aborted:** After a transaction fails, it is rolled back, and the database is restored to its state prior to the start of the transaction.
5. **Committed:** The transaction enters this state after successful completion. The changes are permanently saved in the database.

#### State Transition Diagram

```
graph LR
    A[Active] --> B[Partially Committed]
    A --> C[Failed]
    B --> C
    B --> D[Committed]
    C --> E[Aborted]
    D --> F((Terminated))
    E --> F
```

(Note: If the diagram above does not render, the flow is: Active -> Partially Committed -> Committed. If an error occurs at Active or Partially Committed, it goes to Failed -> Aborted).

### 3. b) Schedule Analysis

**Given Schedule (S):** Based on the table provided  $[[T1, T2, T3], [ReadA], [ReadA], [WriteA], [ReadA], [WriteA]]$ , the chronological order of operations is:

1. **T3:** Read(A)
2. **T2:** Read(A)
3. **T3:** Write(A)
4. **T1:** Read(A)
5. **T1:** Write(A)

### i) Construct the Precedence Graph

To build the precedence graph, we identify **conflicting operations**. A conflict exists if two operations belong to different transactions, access the same data item (A), and at least one is a Write.

#### Conflicts in Schedule S:

1. **T2 Read(A)  $\rightarrow$  T3 Write(A):**
  - T2 reads A at step 2. T3 writes A at step 3.
  - This is a Read-Write conflict.
  - **Edge:**  $T2 \rightarrow T3$
2. **T3 Write(A)  $\rightarrow$  T1 Read(A):**
  - T3 writes A at step 3. T1 reads A at step 4.
  - This is a Write-Read conflict (Reading uncommitted data).
  - **Edge:**  $T3 \rightarrow T1$
3. **T3 Write(A)  $\rightarrow$  T1 Write(A):**
  - T3 writes A at step 3. T1 writes A at step 5.
  - This is a Write-Write conflict (Blind write).
  - **Edge:**  $T3 \rightarrow T1$  (Duplicate edge, ignored for graph structure).

#### Precedence Graph Representation:

- **Nodes:**  $\{T1, T2, T3\}$
- **Edges:**  $\{T2 \rightarrow T3, T3 \rightarrow T1\}$

```
graph LR
    T2((T2)) --> T3((T3))
    T3 --> T1((T1))
```

### ii) Check for Conflict Serializability

**Rule:** A schedule is conflict serializable if and only if its precedence graph is **acyclic** (contains no cycles).

**Analysis:**

1. We have an edge from  $T2$  to  $T3$ .
2. We have an edge from  $T3$  to  $T1$ .
3. The path is linear:  $T2 \rightarrow T3 \rightarrow T1$ .
4. There is no path from  $T1$  back to  $T2$  or  $T3$ .

**Conclusion:** Since there are **no cycles** in the precedence graph, the schedule **IS Conflict Serializable**.

**Equivalent Serial Schedule:** To find the equivalent serial schedule, we perform a **Topological Sort** of the graph:

1. Node with in-degree 0 (no incoming edges): **T2**
2. Remove T2. Remaining Node with in-degree 0: **T3**
3. Remove T3. Remaining Node with in-degree 0: **T1**

**Resulting Serial Schedule:**  $T2 \rightarrow T3 \rightarrow T1$

## DBMS Question 4: Column Family Stores & Scaling Strategies

Marks: 10

Topic: NoSQL Data Models / Distributed Systems

### 4. a) Column Family Stores: Aggregate Data Model

**Definition:** Column Family stores (also known as Wide-Column stores) are a type of NoSQL database that organizes data into rows and columns, but unlike relational databases, the names and format of the columns can vary from row to row in the same table.

#### The Data Model:

- **Row Key:** The unique identifier for a record.
- **Column Family:** A group of related data that is accessed together. Column families are defined upfront (schema-based), but the columns inside them are dynamic.
- **Column:** A pair of (Name, Value).
- **Timestamp:** Used to handle versioning and conflict resolution.

**Aggregate Structure:** In the aggregate data model, the unit of interaction is a collection of related data. Column families allow the database to store the "aggregate" (e.g., a Customer Profile) in a way that allows efficient retrieval of specific subsets of data. For example, a Customer row might have a Profile family and an Orders family. If you only need profile info, the database doesn't need to load the order history.

#### Diagram: Column Family Structure

```
classDiagram
    class Row_Key_101 {
        <<Row Key>>
    }

    class Family_Personal {
        <<Column Family>>
        Col: Name = "Alice"
        Col: Age = 25
    }

    class Family_Contact {
        <<Column Family>>
        Col: Email = "alice@test.com"
        Col: Phone = "555-0199"
    }

    Row_Key_101 *-- Family_Personal
    Row_Key_101 *-- Family_Contact
```

In the diagram above, "Row 101" is the aggregate root. It physically stores the "Personal" data and "Contact" data separately on disk, allowing efficient reads of just one family.

4. b) Sharding vs. Replication

Differentiation

| Feature         | Sharding (Partitioning)                                                                                                        | Replication (Mirroring)                                                                                             |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Definition      | Splitting a large dataset into smaller chunks (shards) and distributing them across multiple servers.                          | Creating multiple copies of the same dataset and storing them on different servers.                                 |
| Primary Goal    | <b>Write Scaling &amp; Storage Capacity.</b> It handles "Big Data" that exceeds the storage/processing power of a single node. | <b>Read Scaling &amp; High Availability.</b> It ensures data survives if a node fails and distributes read traffic. |
| Data Redundancy | No redundancy (by itself). Each piece of data exists on only one shard.                                                        | High redundancy. The same data exists on all replicas.                                                              |
| Failure Impact  | If a shard fails, that specific portion of data becomes unavailable.                                                           | If a replica fails, the system continues to function using other copies.                                            |
| Analogy         | Splitting a phone book: A-M goes to Server 1, N-Z goes to Server 2.                                                            | Printing identical copies of the phone book for everyone to read.                                                   |

Combining Sharding and Replication

To achieve a distributed system that is both **Scalable** (can hold petabytes of data) and **Reliable** (won't lose data if a server crashes), databases combine these two techniques.

How it works:

- 1. **Step 1 (Sharding):** The logical dataset is partitioned based on a Shard Key (e.g., User ID). Users 1-100 go to Shard A, Users 101-200 go to Shard B.
- 2. **Step 2 (Replication):** Each Shard is then treated as a unit and replicated to multiple nodes. Shard A is copied to Node 1 (Primary) and Node 2 (Secondary).

Diagram: Combined Architecture

```
graph TD
    subgraph Logical_Data
        D[Big Dataset]
    end

    subgraph Sharding_Layer
        S1[Shard 1 (Users A-M)]
        S2[Shard 2 (Users N-Z)]
    end

    subgraph Replication_Layer
```

```
N1[Node 1: Shard 1 Primary]
N2[Node 2: Shard 1 Replica]
N3[Node 3: Shard 2 Primary]
N4[Node 4: Shard 2 Replica]
end
```

```
D -->|Partitioning| S1
D -->|Partitioning| S2
```

```
S1 -->|Replicate| N1
S1 -->|Replicate| N2
```

```
S2 -->|Replicate| N3
S2 -->|Replicate| N4
```

### Benefits of Combination:

- **Linear Write Scaling:** Writes for User A go to Shard 1, writes for User N go to Shard 2 (handled in parallel).
- **Fault Tolerance:** If Node 1 crashes, Node 2 takes over Shard 1 without data loss.



# DBMS Question 5: Digital Data & Hadoop

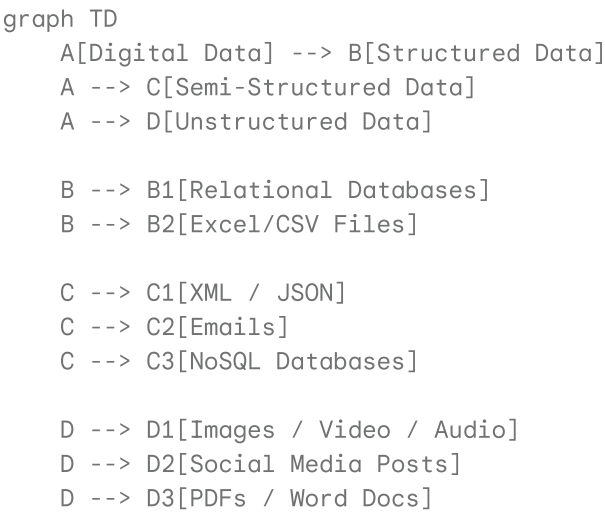
Marks: 10

Topic: Big Data / Hadoop Ecosystem

## 5. a) Classification of Digital Data

Digital data is broadly classified into three categories based on its organization and schema.

### Classification Diagram



### Differentiation: Structured vs. Semi-Structured vs. Unstructured

| Feature          | Structured Data                                                                  | Semi-Structured Data                                                                                | Unstructured Data                                            |
|------------------|----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| Definition       | Data that adheres to a pre-defined data model or schema. It is highly organized. | Data that does not have a formal data model but contains tags or markers to separate data elements. | Data that has no pre-defined structure or organization.      |
| Schema           | <b>Rigid/Static Schema:</b> Structure is defined before data is inserted.        | <b>Flexible/Self-describing:</b> Schema is often contained within the data itself (e.g., tags).     | <b>No Schema:</b> Structure is absent or dynamic.            |
| Storage          | Relational Database Management Systems (RDBMS).                                  | NoSQL databases, XML files, JSON files.                                                             | Data Lakes, Blob storage, File systems.                      |
| Ease of Analysis | Very easy to search and query (using SQL).                                       | Moderately easy; requires parsing (using XQuery, JSON parsers).                                     | Difficult; requires complex processing (Data Mining, AI/ML). |

|          |                                                |                                                    |                                                    |
|----------|------------------------------------------------|----------------------------------------------------|----------------------------------------------------|
| Examples | SQL Tables, Spreadsheets, Transaction records. | HTML code, JSON objects, XML config files, Emails. | Images, Videos, Audio files, Text documents, Logs. |
|----------|------------------------------------------------|----------------------------------------------------|----------------------------------------------------|

## 5. b) Hadoop and Key Considerations

### Definition of Hadoop

**Apache Hadoop** is an open-source software framework used for distributed storage and processing of large datasets (Big Data) across clusters of computers using simple programming models. It is designed to scale up from single servers to thousands of machines.

### Key Considerations of Hadoop

Hadoop is built on specific design philosophies to handle the challenges of Big Data.

#### 1. Commodity Hardware:

- Hadoop does not require expensive, proprietary supercomputers. It is designed to run on clusters of affordable, industry-standard hardware ("commodity hardware").
- Benefit:* Significantly reduces the cost of storing petabytes of data.

#### 2. Fault Tolerance (Replication):

- Since Hadoop runs on commodity hardware, hardware failure is treated as the norm, not the exception.
- Hadoop handles this by replicating data blocks across multiple nodes (default replication factor is 3). If one node fails, the data is available on another.

#### 3. Data Locality:

- In traditional systems, data is moved to the computation unit. In Big Data, moving huge amounts of data is slow (network bottleneck).
- Hadoop moves the **computation to the data**. The processing logic (MapReduce task) runs on the node where the data actually resides.
- Benefit:* Faster processing and reduced network traffic.

#### 4. Scalability (Horizontal Scaling):

- Hadoop scales horizontally. To increase storage or processing power, you simply add more nodes (servers) to the cluster rather than upgrading the existing hardware.

### Diagram: Hadoop Architecture Considerations

```
graph TD
    subgraph Hadoop_Cluster_Considerations
        NN[NameNode (Master)]
        subgraph Rack_1
            DN1[DataNode 1]
            DN2[DataNode 2]
        end
    end
```

```

end

subgraph Rack_2
  DN3[DataNode 3]
  DN4[DataNode 4]
end

end

NN -->|Manages Metadata & Health| DN1
NN -->|Manages Metadata & Health| DN3

DN1 -- "Block A (Replica 1)" --> DN1
DN1 -. ->|Replication (Fault Tolerance)| DN2
DN2 -- "Block A (Replica 2)" --> DN2
DN2 -. ->|Replication (Rack Awareness)| DN3
DN3 -- "Block A (Replica 3)" --> DN3

Task[Computation Task] -->|Data Locality| DN1

style Task fill:#f96,stroke:#333,stroke-width:2px,color:white
style DN1 fill:#bbf,stroke:#333
style DN2 fill:#bbf,stroke:#333
style DN3 fill:#bbf,stroke:#333

```

- **Replication:** Shows Block A copied to Node 2 and Node 3.
- **Data Locality:** Shows the Computation Task being sent directly to DataNode 1 where the data sits.

## DBMS Solution: Weak Entity Analysis

**Scenario:** A bank maintains information about customer accounts and transactions. Each transaction is identified by a transaction number that is unique *only within* the account to which it belongs.

### Solution

#### (i) The Weak Entity

- **Answer: Transaction**
- **Reasoning:** The problem states that the transaction number is unique *only within* the account. It relies on the Account entity for its identity.

#### (ii) The Partial Key of the Weak Entity

- **Answer: Transaction Number**
- **Reasoning:** Since the transaction number distinguishes transactions only within the context of a specific account (e.g., Account A has Trans #1, Account B has Trans #1), it acts as the discriminator or partial key, not a primary key.

#### (iii) The Identifying Relationship

- **Answer: Has** (or *Performed\_On / Belongs\_To*)
- **Reasoning:** This is the relationship that connects the strong entity ( Account ) to the weak entity ( Transaction ). In ER diagrams, this is represented by a double diamond.

#### (iv) The Participating Constraint of the Weak Entity

- **Answer: Total Participation** (Existential Dependence)
- **Reasoning:** A transaction cannot exist without an associated account. Every transaction *must* belong to exactly one account.

#### (v) Justification for Modeling as Weak

- **Justification:** The entity Transaction should be modeled as a weak entity because:
  1. **Lack of Primary Key:** It does not possess sufficient attributes to form a primary key on its own. The Transaction Number is not globally unique (duplicates exist across different accounts).
  2. **Existence Dependency:** A transaction is logically dependent on an account. If an account is deleted, the transaction history associated with it loses its context and validity.
  3. To uniquely identify a specific transaction, we need the Primary Key of the owner entity ( Account Number ) combined with the Partial Key of the weak entity

( Transaction Number ).

## ER Diagram Visualization

Below is the Entity-Relationship representation of the solution:

```
erDiagram
    ACCOUNT ||--}| TRANSACTION : "Has / Maintains"

    ACCOUNT {
        string Account_Number PK
        string Customer_Name
        float Balance
    }

    TRANSACTION {
        int Transaction_Number UK "Partial Key (Discriminator)"
        date Date
        float Amount
        string Type
    }
```

- **ACCOUNT:** Strong Entity (Independent).
- **TRANSACTION:** Weak Entity (Dependent on Account).
- **Relationship:** One Account can have Many Transactions (1:N), but a Transaction belongs to exactly one Account.

## DBMS Part A: 24.1.26 Short Answer Solutions

1. Briefly distinguish between a database schema and a database state with one key difference.

- **Database Schema (Intension):** This is the logical design or description of the database structure (e.g., table names, data types, constraints). It is defined during the design phase and changes very infrequently.
- **Database State (Extension):** This refers to the actual data stored in the database at a specific moment in time. It changes every time a record is inserted, updated, or deleted.
- **Key Difference: Frequency of Change.** The schema is relatively static (structure), while the state changes frequently (content).

2. In a university database, the Enrollment table records the courses taken by students... Identify the attribute that serves as a partial key and briefly justify your choice.

- **Partial Key:** The combination of {CourseID, Semester}.
- **Justification:** If we model Enrollment as a weak entity dependent on the Student entity, the StudentID (from the owner entity) is needed for global identification. Within the context of a specific student, we need to distinguish which specific enrollment we are talking about. Since a student can enroll in many courses, and potentially in different semesters, neither CourseID nor Semester alone is unique for a student. The combination of CourseID and Semester distinguishes one enrollment record from another for the same student.
  - *Note: While the question asks for "the attribute" (singular), a partial key is often a composite set of attributes in M:N relationships modeled as weak entities.*

3. A small neighborhood bakery uses a spreadsheet to record daily sales and inventory. Should it adopt a DBMS? Justify your answer briefly.

- **Answer: No,** it likely should not adopt a full DBMS.
- **Justification:** For a "small neighborhood bakery," the data volume is low, the operations are simple (flat lists of sales), and there is likely only a single user accessing the data. The cost, complexity, and maintenance overhead of a DBMS would outweigh the benefits. A spreadsheet is sufficient, cheaper, and easier to use for this scale.

4. Differentiate between candidate key and super keys with examples for each.

- **Super Key:** A set of one or more attributes that can uniquely identify a tuple in a relation. It may contain redundant attributes.
  - *Example:* In a Student table, {StudentID, Name} is a Super Key.
- **Candidate Key:** A minimal Super Key. It is a Super Key from which no attribute can be removed without losing the uniqueness property.
  - *Example:* {StudentID} is a Candidate Key. (Removing 'StudentID' leaves 'Name', which is not unique).

5. When a record is deleted from a parent table that is still referenced by a child table, which database constraint is violated? Briefly explain why this violation occurs.

- **Constraint Violated: Referential Integrity Constraint** (or Foreign Key Constraint).
- **Explanation:** This constraint ensures that a foreign key value in a child table must always point to a valid, existing primary key in the parent table. If you delete the parent record while child records still point to it, those child records become "orphans" pointing to a non-existent value, thus violating the integrity of the reference.